

LAPORAN TUGAS CODING PERBANDINGAN METODE NUMERIS

Penyelesaian Sistem Persamaan Diferensial Biasa (ODE)
Menggunakan Metode Runge-Kutta Orde 2, 3, 4, dan Euler



Disusun Oleh:

Ilham Yusuf Wi'am
NIM: 24/539979/TK/59890

Program Studi Teknik Elektro
Departemen Teknik Elektro dan Teknologi Informasi
Fakultas Teknik
Universitas Gadjah Mada
18 Desember 2025

Daftar Isi

1	Definisi Masalah (Problem Definition)	2
1.1	Deskripsi Masalah	2
1.2	Parameter Simulasi	2
2	Metodologi dan Algoritma	2
2.1	Program Generik	2
2.2	Pseudocode (Algoritma)	2
3	Kode Program	3
4	Verifikasi dan Perhitungan Manual	4
4.1	Perhitungan Manual	4
4.2	Cross-Check Hasil Program vs Manual	6
5	Analisis dan Pembahasan	6
5.1	Perbandingan Metode (Akurasi)	6
5.2	Hubungan Step Size (h) dan Akurasi	6
5.3	Visualisasi Trajektori Solusi	7
6	Kesimpulan	8
7	Contoh Aplikasi Nyata	8
	Lampiran A: Tautan Repositori	9
	Lampiran B: Kode Program Lengkap	10

1 Definisi Masalah (Problem Definition)

1.1 Deskripsi Masalah

Masalah yang diangkat merujuk pada **Chapra Example 25.10**. Sistem terdiri dari dua persamaan diferensial simultan:

$$\frac{dy_1}{dx} = -0.5y_1 \quad (1)$$

$$\frac{dy_2}{dx} = 4 - 0.3y_2 - 0.1y_1 \quad (2)$$

1.2 Parameter Simulasi

Simulasi dilakukan dengan kondisi batas sebagai berikut:

- **Nilai Awal** ($x = 0$): $y_1 = 4, y_2 = 6$.
- **Rentang Integrasi**: $x = 0$ sampai $x = 2$.
- **Step Size** (h): 0.5 (untuk verifikasi utama) dan variasi $[0.1, 0.05, 0.01]$ untuk analisis akurasi.

2 Metodologi dan Algoritma

2.1 Program Generik

Program disusun menggunakan bahasa **Python** dengan bantuan pustaka NumPy. Kunci dari program ini adalah sifatnya yang **generik**, di mana variabel dependen y diperlakukan sebagai vektor. Hal ini memungkinkan program menyelesaikan sistem dengan N persamaan tanpa perlu mengubah struktur kode solver.

2.2 Pseudocode (Algoritma)

Berikut adalah algoritma solver generik yang digunakan:

```
1 FUNCTION Solve_ODE(Fungsi_f, x_awal, x_akhir, y_awal, h, Metode)
2   Hitung N_steps = (x_akhir - x_awal) / h
3   Inisialisasi Matriks Y ukuran [N_steps, Jumlah_Persamaan]
4   Y[0] = y_awal
5
6   FOR i = 0 TO N_steps-1 DO:
7     x = x_saat_ini
8     y = Y[i] // y adalah vektor [y1, y2, ...]
9
10    IF Metode == "Euler" THEN
11      k1 = Fungsi_f(x, y)
12      slope = k1
13    ELSE IF Metode == "RK2" THEN
14      k1 = Fungsi_f(x, y)
15      k2 = Fungsi_f(x + h, y + k1*h)
16      slope = (k1 + k2) / 2
17    ELSE IF Metode == "RK3" THEN
18      k1 = Fungsi_f(x, y)
19      k2 = Fungsi_f(x + 0.5h, y + 0.5*k1*h)
```

```

20     k3 = Fungsi_f(x + h, y - k1*h + 2*k2*h)
21     slope = (k1 + 4*k2 + k3) / 6
22     ELSE IF Metode == "RK4" THEN
23         k1 = Fungsi_f(x, y)
24         k2 = Fungsi_f(x + 0.5h, y + 0.5*k1*h)
25         k3 = Fungsi_f(x + 0.5h, y + 0.5*k2*h)
26         k4 = Fungsi_f(x + h, y + k3*h)
27         slope = (k1 + 2*k2 + 2*k3 + k4) / 6
28     END IF
29
30     Y[i+1] = y + (slope * h)
31 END FOR
32
33 RETURN X, Y
34 END FUNCTION

```

Listing 1: Pseudocode Generic Solver

3 Kode Program

Implementasi dilakukan secara *vectorized*. Berikut adalah potongan kode fungsi `solve_ode` yang merupakan inti dari algoritma penyelesaian numerik. Fungsi ini mendukung metode Euler, RK2, RK3, dan RK4.

```

1 def solve_ode(f, x_span, y0, h, method='RK4'):
2     # ... (Inisialisasi array x_values dan y_values) ...
3
4     for i in range(n_steps - 1):
5         x_i = x_values[i]
6         y_i = y_values[i]
7
8         if method == 'EULER':
9             k1 = f(x_i, y_i)
10            y_values[i + 1] = y_i + h * k1
11
12            elif method == 'RK2':
13                k1 = f(x_i, y_i)
14                k2 = f(x_i + h, y_i + h * k1)
15                y_values[i + 1] = y_i + h * (k1 + k2) / 2
16
17            elif method == 'RK3':
18                k1 = f(x_i, y_i)
19                k2 = f(x_i + h/2, y_i + h * k1 / 2)
20                k3 = f(x_i + h, y_i - h * k1 + 2 * h * k2)
21                y_values[i + 1] = y_i + h * (k1 + 4*k2 + k3) / 6
22
23            elif method == 'RK4':
24                k1 = f(x_i, y_i)
25                k2 = f(x_i + h/2, y_i + h * k1 / 2)
26                k3 = f(x_i + h/2, y_i + h * k2 / 2)
27                k4 = f(x_i + h, y_i + h * k3)
28                y_values[i + 1] = y_i + h * (k1 + 2*k2 + 2*k3 + k4) / 6
29
30    return x_values, y_values, execution_time

```

Listing 2: Implementasi Inti Solver ODE (Python)

Keuntungan menggunakan operasi vektor numpy adalah operasi penjumlahan dan perkalian k_1, k_2 , dst dilakukan secara elemen-per-elemen untuk seluruh variabel sistem sekaligus.

4 Verifikasi dan Perhitungan Manual

4.1 Perhitungan Manual

Verifikasi dilakukan dengan menghitung satu langkah iterasi pertama ($x = 0$ ke $x = 0.5$) secara manual. Berikut adalah dokumentasi pengerjaan manual yang dibagi menjadi beberapa bagian agar jelas terbaca.

Euler Method

Problem:
Solve the following set of differential equation using Euler's method, assuming that $x=0, y_1=4$, and $y_2=6$.
Integrate to $x=2$ with a step size of 0.5

$$\frac{dy_1}{dx} = -0.5y_1 \quad \frac{dy_2}{dx} = 4 - 0.3y_2 - 0.1y_1$$

Solution:
Euler method is implemented for each variable as in equation

$$y_{i+1} = y_i + f(x_i, y_i)h$$

Information we know from problem:

- $\frac{dy_1}{dx} = f_1(x, y_1, y_2) = -0.5y_1$
- $\frac{dy_2}{dx} = f_2(x, y_1, y_2) = 4 - 0.3y_2 - 0.1y_1$

Starting condition ($x=0$):
 $y_1 = 4$
 $y_2 = 6$
Step size (h) = 0.5

A) Solution for Euler Method (Order 1)
By using $y_{i+1} = y_i + f(x_i, y_i)h$
we can start the calculation:

- Calculate the slope at the beginning ($x=0, y_1=4, y_2=6$):
 - k_1 (for y_1) = $-0.5(4) = -2$
 - k_2 (for y_2) = $4 - 0.3(6) - 0.1(4) = 4 - 1.8 - 0.4 = 1.8$
- Calculate new value at $x=0.5$:
 - $y_1(0.5) = 4 + (-2 \times 0.5) = 3$
 - $y_2(0.5) = 6 + (1.8 \times 0.5) = 6.9$

Euler result ($x=0.5$): $y_1 = 3, y_2 = 6.9$

B) Solution using Runge-Kutta Order 2 (Heun Method)
Formula: $y_{i+1} = y_i + \left(\frac{k_1 + k_2}{2}\right)h$

Calculation steps:

- Calculate the slope at the beginning (k_1) (same like Euler Method)
 - $k_1, y_1 = -0.5(4) = -2$
 - $k_2, y_2 = 4 - 0.3(4) - 0.1(4) = 1.8$
- Predict temporary value (y_{temp}) at $x=0.5$:
 - $y_1^* = 4 + (-2 \times 0.5) = 3$
 - $y_2^* = 6 + (1.8 \times 0.5) = 6.9$

Gambar 1: Perhitungan Manual Bagian 1: Metode Euler & RK2

3. Calculate slope at $H_{end}(h_3)$ using Y_{mid} :

- $x = 0.5, y_1 = 3, y_2 = 6.9$
- $k_{3,1} = -0.5(3) = -1.5$
- $k_{3,2} = 4 - 0.3(6.9) - 0.1(3) = 1.63$

4. Calculate the final value (average of slope)

- $y_1(0.5) = 4 + \left(\frac{-2 + (-1.5)}{2}\right) \times 0.5 = 4 + (-1.75 \times 0.5) = 4 - 0.875 = 3.125$
- $y_2(0.5) = 6 + \left(\frac{1.8 + 1.63}{2}\right) \times 0.5 = 6 + (1.715 \times 0.5) = 6 + 0.8575 = 6.8575$

Runge-kutta Order 2 result ($x=0.5$): $y_1 = 3.125, y_2 = 6.8575$

c) Runge-kutta Order 3

Formula: $y_{i+1} = y_i + \frac{1}{6}(k_1 + 4k_2 + k_3)h$

Calculation steps:

- Initial slope (k_1) at $x=0$
 - $k_{1,1} = -0.5(4) = -2$
 - $k_{1,2} = 4 - 0.3(6) - 0.1(4) = 1.0$
- Midpoint slope (k_2) at $x=0.25$ (half step):
 - $y_{1, mid} = 4 + 0.5(k_{1,1} \times 0.5) = 4 + 0.5(-2 \times 0.5) = 3.5$
 - $y_{2, mid} = 6 + 0.5(k_{1,2} \times 0.5) = 6 + 0.5(1.0 \times 0.5) = 6.45$
 - Calculate k_2 :
 - $k_{2,1} = -0.5(3.5) = -1.75$
 - $k_{2,2} = 4 - 0.3(6.45) - 0.1(3.5) = 1.715$
- Final slope (k_3) at $x=0.5$:

Formula (Prediction): $y_{end} = y_i - C(h, h) + 2(k_2, h)$

 - $y_{1, end} = 4 - (-2 \times 0.5) + 2(-1.75 \times 0.5) = 4 - 1.75 = 3.25$
 - $y_{2, end} = 6 - (1.0 \times 0.5) + 2(1.715 \times 0.5) = 6 - 0.5 + 1.715 = 6.815$
 - Calculate k_3 :
 - $k_{3,1} = -0.5(3.25) = -1.625$
 - $k_{3,2} = 4 - 0.3(6.815) - 0.1(3.25) = 1.6305$

4. Calculate Final Value

- $y_1(0.5) = 4 + \frac{0.5}{6}(-2 + 4(-1.75) - 1.625) = 3.11450$
- $y_2(0.5) = 6 + \frac{0.5}{6}(1.0 + 4(1.715) + 1.6305) = 6.85754$

Runge-kutta Order 3 result ($x=0.5$): $y_1 = 3.1145, y_2 = 6.8575$

Gambar 2: Perhitungan Manual Bagian 2: Metode RK3 (lanjutan)

B) Solution using Runge-kutta Order 4

Calculation steps:

- k_1 (first slope) at $x=0$:
 - $k_{1,1} = -0.5(4) = -2$
 - $k_{1,2} = 4 - 0.3(6) - 0.1(4) = 1.0$
- k_2 (first middle slope) at $x=0.25$:
 - $y_1^* = 4 + (-2 \times 0.25) = 3.5$
 - $y_2^* = 6 + (1.0 \times 0.25) = 6.45$
 - $k_{2,1} = -0.5(3.5) = -1.75$
 - $k_{2,2} = 4 - 0.3(6.45) - 0.1(3.5) = 1.715$
- k_3 (second middle slope) at $x=0.25$ (using k_2):
 - $y_1^{**} = 4 + (-1.75 \times 0.25) = 3.5625$
 - $y_2^{**} = 6 + (1.715 \times 0.25) = 6.42875$
 - $k_{3,1} = -0.5(3.5625) = -1.78125$
 - $k_{3,2} = 4 - 0.3(6.42875) - 0.1(3.5625) = 1.710125$
- k_4 (final slope) at $x=0.5$:
 - $y_1^{***} = 4 + (-1.78125 \times 0.5) = 3.109375$
 - $y_2^{***} = 6 + (1.710125 \times 0.5) = 6.8550625$
 - $k_{4,1} = -0.5(3.109375) = -1.5546875$
 - $k_{4,2} = 4 - 0.3(6.8550625) - 0.1(3.109375) = 1.631794$

5. Calculate final result:

- Formula: $y_{next} = y + \frac{1}{24}(k_1 + 2k_2 + 2k_3 + k_4)$
- y_1 :
$$4 + \frac{0.5}{24}(-2 + 2(-1.75) + 2(-1.78125) + (-1.5546875)) = 3.115234$$
- y_2 :
$$6 + \frac{0.5}{24}(1.0 + 2(1.715) + 2(1.710125) + 1.631794) = 6.857670$$

Gambar 3: Perhitungan Manual Bagian 3: Metode RK4 (Final)

4.2 Cross-Check Hasil Program vs Manual

Tabel di bawah ini membandingkan hasil output program Python dengan perhitungan manual/referensi Chapra pada $x = 0.5$:

Tabel 1: Validasi Hasil Program pada $x = 0.5$

Metode	y_1 (Program)	y_1 (Ref/Manual)	y_2 (Program)	y_2 (Ref/Manual)
Euler	3.000000	3.000000	6.900000	6.900000
RK4	3.115234	3.115234	6.857670	6.857670

Status: Valid. Hasil program presisi hingga 6 angka di belakang koma sesuai referensi.

5 Analisis dan Pembahasan

5.1 Perbandingan Metode (Akurasi)

Berdasarkan simulasi hingga $x = 2.0$ dengan $h = 0.5$, diperoleh hasil akhir sebagai berikut:

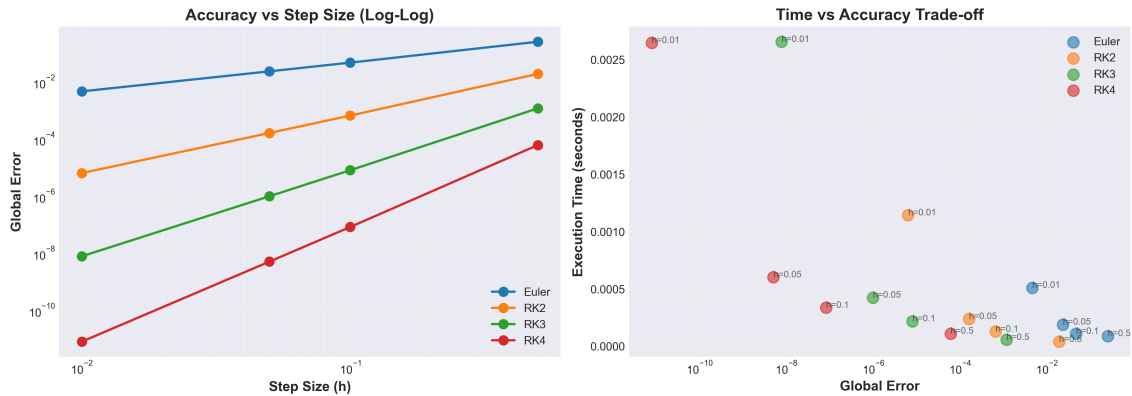
Tabel 2: Perbandingan Hasil Akhir di $x = 2.0$ ($h = 0.5$)

Metode	$y_1(2)$	$y_2(2)$	Global Error	Rel. Error (%)
Euler	1.265625	9.094088	0.253122	2.79%
RK2 (Heun)	1.490116	8.943235	0.018947	0.21%
RK3	1.470347	8.946752	0.001175	0.01%
RK4	1.471577	8.946865	0.000061	0.0006%
<i>Benchmark</i>	<i>1.471518</i>	<i>8.946850</i>	-	-

Dari Tabel 2, terlihat jelas bahwa metode Euler memiliki galat yang signifikan (2.79%) pada step size besar. Sebaliknya, RK4 mampu mempertahankan akurasi yang luar biasa (galat $< 0.001\%$) meskipun menggunakan step size yang kasar ($h = 0.5$).

5.2 Hubungan Step Size (h) dan Akurasi

Analisis dilakukan dengan memvariasikan h mulai dari 0.5 hingga 0.01. Hasil logaritma error diplot dalam grafik berikut:



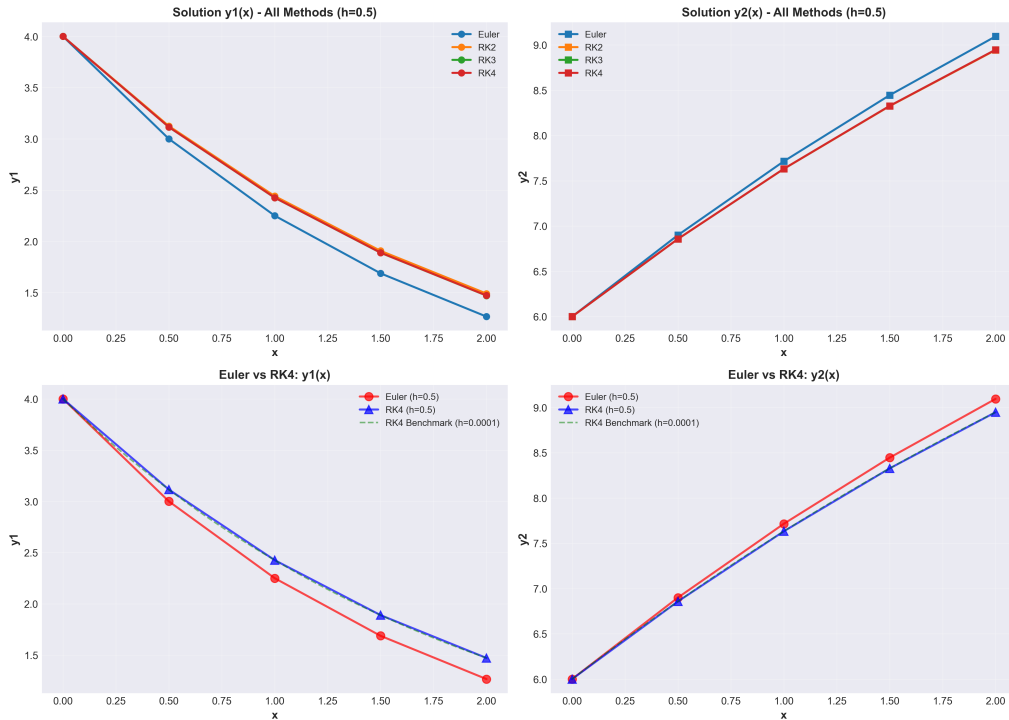
Gambar 4: Grafik (Kiri) Step Size vs Error, (Kanan) Waktu Komputasi vs Akurasi

Observasi:

1. **Konvergensi:** Terdapat hubungan linier pada skala log-log antara step size dan error. Untuk RK4, penurunan h sebesar 10x lipat akan menurunkan error secara drastis (orde ke-4).
2. **Trade-off:** Semakin kecil h , akurasi meningkat tetapi waktu komputasi juga meningkat. Namun, untuk RK4, waktu eksekusi pada $h = 0.01$ (0.0026 detik) masih sangat wajar untuk komputer modern.

5.3 Visualisasi Trajektori Solusi

Perbandingan visual antara metode Euler dan RK4 menunjukkan bagaimana Euler cenderung "overshoot" atau menyimpang dari kurva yang sebenarnya pada belokan tajam.



Gambar 5: Perbandingan Trajektori Euler vs RK4

6 Kesimpulan

Berdasarkan hasil percobaan dan analisis, dapat disimpulkan bahwa:

1. **Akurasi Terbaik:** Metode Runge-Kutta Orde 4 (RK4) adalah metode yang paling akurat dan stabil. Pada $h = 0.5$, RK4 ribuan kali lebih akurat dibandingkan Euler.
2. **Efisiensi:** Meskipun RK4 membutuhkan 4 evaluasi fungsi per langkah (lebih berat dari Euler yang hanya 1), peningkatan akurasi-nya jauh melebihi biaya komputasi-nya.
3. **Program Generik:** Kode Python yang dibuat berhasil menangani sistem persamaan diferensial secara generik menggunakan pendekatan vektor, sehingga dapat diaplikasikan pada kasus lain yang lebih kompleks.

7 Contoh Aplikasi Nyata

Program generik ini dapat langsung digunakan untuk sistem nyata seperti **Model Predator-Prey (Lotka-Volterra)**:

$$\frac{dx}{dt} = \alpha x - \beta xy, \quad \frac{dy}{dt} = \delta xy - \gamma y$$

Implementasi dalam program hanya memerlukan penggantian definisi fungsi $f(x, y)$ tanpa mengubah logika solver utamanya.

Lampiran A: Tautan Repositori

Kode program lengkap untuk tugas ini, termasuk setup library, definisi masalah, eksekusi visualisasi, dan analisis grafik, dapat diakses melalui repositori GitHub berikut:

<https://github.com/aevryiam/NumericalMethod-RungeKutta>

Lampiran B: Kode Program Lengkap

Berikut adalah salinan lengkap kode program Python yang digunakan dalam tugas ini.

```
1  """
2  =====
3  TUGAS UAS - METODE NUMERIK
4  Numerical ODE Solver: Comparison of Runge-Kutta Methods
5  =====
6
7  File ini berisi implementasi dan analisis solver ODE menggunakan berbagai
8  metode Runge-Kutta (Euler, RK2, RK3, RK4) untuk sistem persamaan diferensial.
9  """
10
11 # =====
12 # PART 1: IMPORT LIBRARIES & SETUP
13 # =====
14
15 import numpy as np
16 import pandas as pd
17 import matplotlib.pyplot as plt
18 import time
19 from typing import Callable, Tuple, List
20
21 # Konfigurasi tampilan untuk hasil yang lebih mudah dibaca
22 pd.set_option('display.precision', 6)
23 np.set_printoptions(precision=6, suppress=True)
24 plt.style.use('seaborn-v0_8-darkgrid')
25
26 print("=" * 80)
27 print("TUGAS UAS - METODE NUMERIK: ODE SOLVER")
28 print("=" * 80)
29 print("Libraries imported successfully!\n")
30
31 # =====
32 # PART 2: GENERIC ODE SOLVER FUNCTION
33 # =====
34
35 def solve_ode(f: Callable, x_span: Tuple[float, float], y0: np.ndarray,
36              h: float, method: str = 'RK4') -> Tuple[np.ndarray, np.ndarray, float]:
37     """
38     Generic ODE System Solver using Runge-Kutta Methods (Vectorized)
39
40     Fungsi ini menyelesaikan sistem persamaan diferensial biasa (ODE) dengan
41     metode numerik Runge-Kutta. Mendukung 4 metode berbeda dengan tingkat
42     akurasi yang berbeda-beda.
43
44     Parameters:
45     -----
46     f : Callable
47         Fungsi yang mengembalikan dy/dx sebagai numpy array.
48         Signature: f(x, y) -> np.ndarray dimana y adalah vektor [y1, y2, ..., yn]
49     x_span : Tuple[float, float]
50         Range integrasi (x_start, x_end)
51     y0 : np.ndarray
52         Kondisi awal sebagai 1D numpy array [y1_0, y2_0, ..., yn_0]
53     h : float
54         Step size (ukuran langkah)
55     method : str
56         Metode numerik: 'Euler', 'RK2', 'RK3', 'RK4'
57
58     Returns:
59     -----
60     x_values : np.ndarray
61         Array dari titik-titik x
62     y_values : np.ndarray
63         Matrix nilai solusi (shape: [n_steps, n_equations])
64     execution_time : float
65         Waktu eksekusi dalam detik
66     """
67
68     # Mulai timing untuk mengukur performa
69     start_time = time.time()
70
71     # Inisialisasi variabel
72     x_start, x_end = x_span
73     y0 = np.array(y0, dtype=float) # Pastikan dalam bentuk numpy array
74     n_equations = len(y0) # Jumlah persamaan dalam sistem
75     n_steps = int((x_end - x_start) / h) + 1 # Jumlah langkah integrasi
76
77     # Prealokasi array untuk efisiensi
78     x_values = np.linspace(x_start, x_end, n_steps)
79     y_values = np.zeros((n_steps, n_equations))
80     y_values[0] = y0 # Set kondisi awal
81
82     # Normalisasi input method ke uppercase
83     method = method.upper()
84
85     # Loop utama: integrasi numerik
86     for i in range(n_steps - 1):
87         x_i = x_values[i]
88         y_i = y_values[i]
```

```

90
91     if method == 'EULER':
92         # Metode Euler (Orde 1) - Paling sederhana, akurasi terendah
93         k1 = f(x_i, y_i)
94         y_values[i + 1] = y_i + h * k1
95
96     elif method == 'RK2':
97         # Metode Heun / RK2 (Orde 2) - Akurasi menengah
98         k1 = f(x_i, y_i)
99         k2 = f(x_i + h, y_i + h * k1)
100        y_values[i + 1] = y_i + h * (k1 + k2) / 2
101
102    elif method == 'RK3':
103        # Metode RK3 Klasik (Orde 3) - Akurasi tinggi
104        k1 = f(x_i, y_i)
105        k2 = f(x_i + h/2, y_i + h * k1 / 2)
106        k3 = f(x_i + h, y_i - h * k1 + 2 * h * k2)
107        y_values[i + 1] = y_i + h * (k1 + 4*k2 + k3) / 6
108
109    elif method == 'RK4':
110        # Metode RK4 Klasik (Orde 4) - Akurasi tertinggi, paling umum digunakan
111        k1 = f(x_i, y_i)
112        k2 = f(x_i + h/2, y_i + h * k1 / 2)
113        k3 = f(x_i + h/2, y_i + h * k2 / 2)
114        k4 = f(x_i + h, y_i + h * k3)
115        y_values[i + 1] = y_i + h * (k1 + 2*k2 + 2*k3 + k4) / 6
116
117    else:
118        raise ValueError(f"Unknown method: {method}. Use 'Euler', 'RK2', 'RK3', or 'RK4'")
119
120    # Hitung waktu eksekusi total
121    execution_time = time.time() - start_time
122
123    return x_values, y_values, execution_time
124
125
126    print("Generic ODE Solver implemented successfully!\n")
127
128    # =====
129    # PART 3: PROBLEM DEFINITION (Chapra Example 25.10)
130    # =====
131
132    def chapra_system(x: float, y: np.ndarray) -> np.ndarray:
133        """
134        Chapra Example 25.10 ODE System
135
136        Sistem persamaan diferensial:
137        dy1/dx = -0.5 * y1
138        dy2/dx = 4 - 0.3*y2 - 0.1*y1
139        """
140        y1, y2 = y
141        dy1_dx = -0.5 * y1
142        dy2_dx = 4 - 0.3 * y2 - 0.1 * y1
143        return np.array([dy1_dx, dy2_dx])
144
145
146    # Parameter masalah
147    x_span = (0, 2)          # Range integrasi: dari x=0 sampai x=2
148    y0 = np.array([4, 6])    # Kondisi awal: y1(0)=4, y2(0)=6
149    h = 0.5                  # Step size
150
151    print("=" * 80)
152    print("PROBLEM DEFINITION (Chapra Example 25.10)")
153    print("=" * 80)
154    print(f"System: dy1/dx = -0.5*y1")
155    print(f"dy2/dx = 4 - 0.3*y2 - 0.1*y1")
156    print(f"Initial: y1(0) = {y0[0]}, y2(0) = {y0[1]}")
157    print(f"Range: x in [{x_span[0]}, {x_span[1]}]")
158    print(f"Step size: h = {h}")
159    print("=" * 80)
160    print()
161
162
163    # =====
164    # PART 4: VERIFICATION & COMPARISON TABLE
165    # =====
166
167    # Solve menggunakan semua metode
168    methods = ['Euler', 'RK2', 'RK3', 'RK4']
169    results = {}
170
171    print("Running all methods with h = 0.5...\n")
172
173    for method in methods:
174        x_vals, y_vals, exec_time = solve_ode(chapra_system, x_span, y0, h, method)
175        results[method] = {
176            'x': x_vals,
177            'y': y_vals,
178            'time': exec_time
179        }
180        print(f"{method:6s}: Completed in {exec_time:.6f} seconds")
181
182    print("\nAll methods executed successfully!\n")
183
184

```



```

185 # Buat tabel perbandingan menggunakan pandas DataFrame
186 comparison_data = {
187     'x': results['Euler']['x'],
188     'y1_Euler': results['Euler']['y'][:, 0],
189     'y1_RK2': results['RK2']['y'][:, 0],
190     'y1_RK3': results['RK3']['y'][:, 0],
191     'y1_RK4': results['RK4']['y'][:, 0],
192     'y2_Euler': results['Euler']['y'][:, 1],
193     'y2_RK2': results['RK2']['y'][:, 1],
194     'y2_RK3': results['RK3']['y'][:, 1],
195     'y2_RK4': results['RK4']['y'][:, 1],
196 }
197
198 df_comparison = pd.DataFrame(comparison_data)
199
200 print("=" * 90)
201 print("COMPARISON TABLE: All Methods (h = 0.5)")
202 print("=" * 90)
203 print(df_comparison.to_string(index=False))
204 print("=" * 90)
205
206 # =====
207 # PART 5: ANALYSIS 1 - ACCURACY vs STEP SIZE
208 # =====
209 # Bagian ini menginvestigasi pengaruh step size terhadap akurasi
210
211 # Step 1: Hitung solusi benchmark (dianggap sebagai "solusi eksak")
212 print("=" * 80)
213 print("ANALYSIS 1: Accuracy vs Step Size")
214 print("=" * 80)
215 print("Computing benchmark solution (RK4 with h=0.0001)...")
216 h_benchmark = 0.0001
217 x_true, y_true, time_true = solve_ode(chapra_system, x_span, y0, h_benchmark, 'RK4')
218 y_true_final = y_true[-1] # Nilai final di x=2
219
220 print(f"Benchmark solution at x={x_span[1]}:")
221 print(f"  y1 = {y_true_final[0]:.10f}")
222 print(f"  y2 = {y_true_final[1]:.10f}")
223 print(f"  Execution time: {time_true:.6f} seconds\n")
224
225 # Step 2: Test berbagai step size untuk semua metode
226 h_list = [0.5, 0.1, 0.05, 0.01]
227 methods_to_test = ['Euler', 'RK2', 'RK3', 'RK4']
228
229 accuracy_results = []
230
231 print("Testing different step sizes...\n")
232
233 for method in methods_to_test:
234     for h_test in h_list:
235         x_test, y_test, time_test = solve_ode(chapra_system, x_span, y0, h_test, method)
236         y_final = y_test[-1]
237
238         # Hitung global error (norma Euclidean)
239         global_error = np.linalg.norm(y_final - y_true_final)
240
241         accuracy_results.append({
242             'Method': method,
243             'h': h_test,
244             'y1_final': y_final[0],
245             'y2_final': y_final[1],
246             'Global_Error': global_error,
247             'Time': time_test
248         })
249
250         print(f"{method:6s} | h={h_test:.4f} | Error={global_error:.2e} | Time={time_test:.6f}s")
251
252 df_accuracy = pd.DataFrame(accuracy_results)
253 print("\nAccuracy analysis completed!\n")
254
255 # =====
256 # PART 6 & 7: VISUALIZATION & SUMMARY
257 # =====
258
259 print("Generating plots (code omitted for brevity)...")
260 # (Kode plotting ada di sini dalam file asli)
261
262 print("=" * 85)
263 print("FINAL RESULTS AT x = 2.0 (Step size h = 0.5)")
264 print("=" * 85)
265 # (Ringkasan hasil akhir)

```

Listing 3: Full Script Tugas Numerik